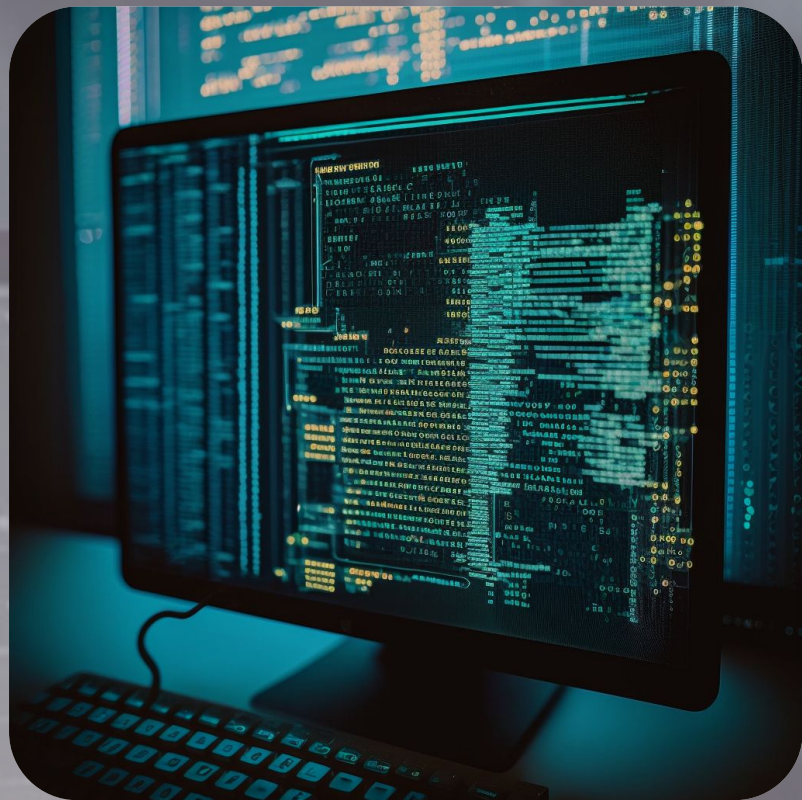
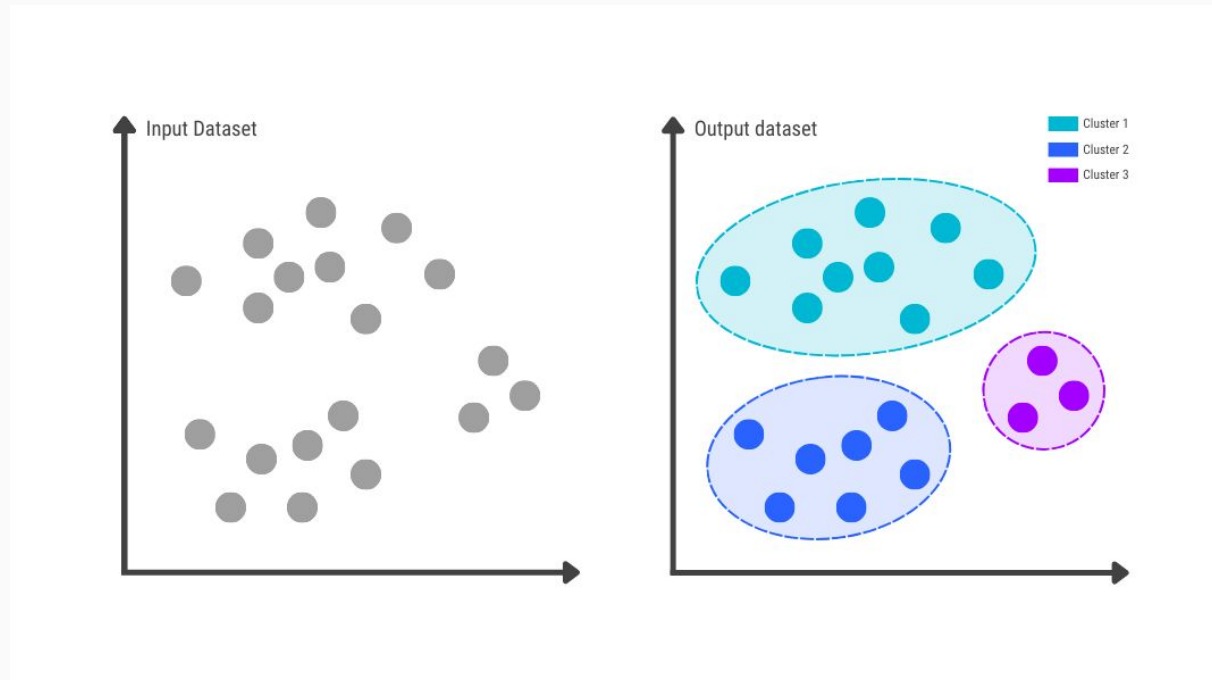




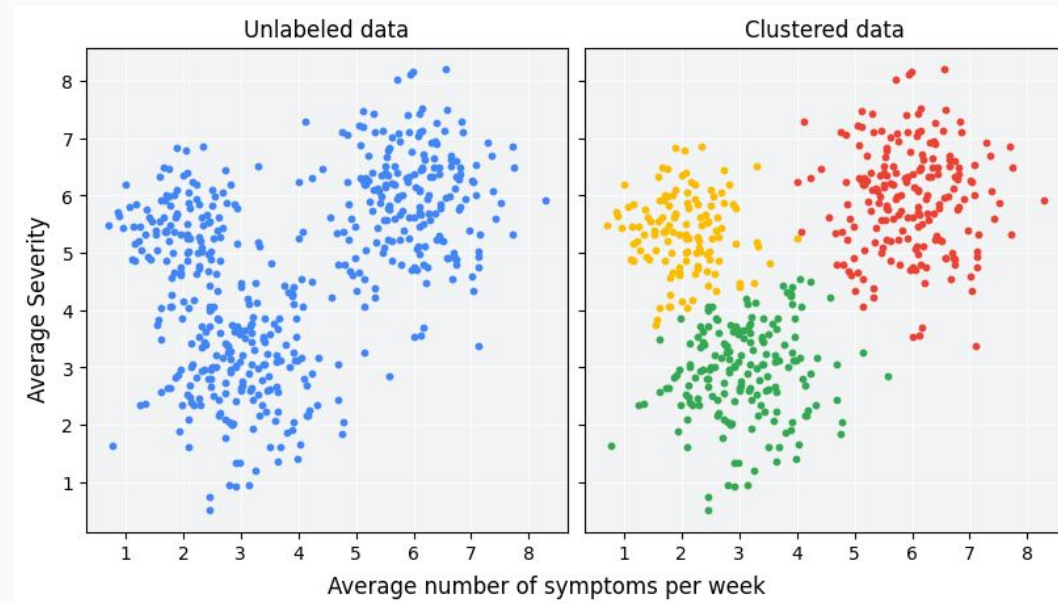
# INTRODUCCIÓN A MODELOS DE CLUSTERING

# ¿Qué es Clustering?

Clustering es una técnica de aprendizaje no supervisado que tiene como objetivo agrupar datos similares entre sí dentro del mismo grupo o cluster, y separar los diferentes entre grupos distintos.



A diferencia de los modelos supervisados, no se utilizan etiquetas (targets). El modelo descubre patrones y estructuras ocultas en los datos por sí mismo.



- ❑ Detectar estructura interna de los datos
- ❑ Agrupar observaciones que comparten características comunes
- ❑ Explorar o resumir grandes volúmenes de datos
- ❑ Preprocesamiento para modelos supervisados (por ejemplo, segmentación previa)

# ¿Para qué sirve el clustering?



## Segmentación de clientes

Agrupar clientes según hábitos de compra

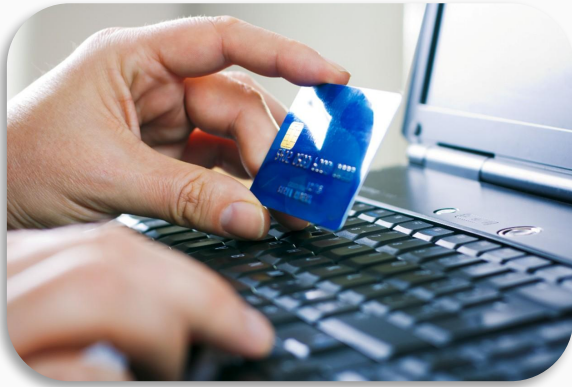


## Agrupación de documentos

Clasificar artículos sin etiquetas previas



# ¿Para qué sirve el clustering?



## Detección de anomalías

Identificar comportamientos inusuales en transacciones



## Agrupación genética

Clasificar especies o muestras biológicas por expresión genética

- No requiere etiquetas o salidas
- Los grupos no están predefinidos
- Los resultados dependen fuertemente de:
  - ✓ La métrica de distancia
  - ✓ El algoritmo utilizado
  - ✓ Los hiperparámetros elegidos (como el número de clusters)

# Tipos comunes de clustering

| Tipo                                              | ¿Qué hace?                                                        |
|---------------------------------------------------|-------------------------------------------------------------------|
| <b>Particional (K-means)</b>                      | Divide el conjunto de datos en $k$ grupos con centroides          |
| <b>Jerárquico</b>                                 | Crea una estructura de árbol (dendrograma) con subgrupos anidados |
| <b>Basado en densidad (DBSCAN)</b>                | Encuentra grupos en zonas densas del espacio, ignora el ruido     |
| <b>Basado en similitud (Affinity Propagation)</b> | Propaga similitudes entre puntos para decidir agrupaciones        |

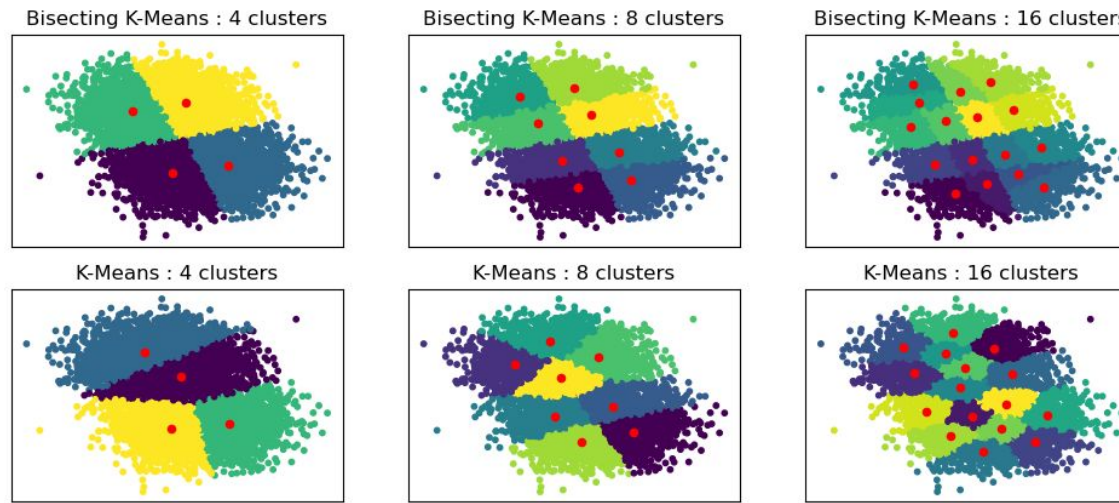




# K-Means

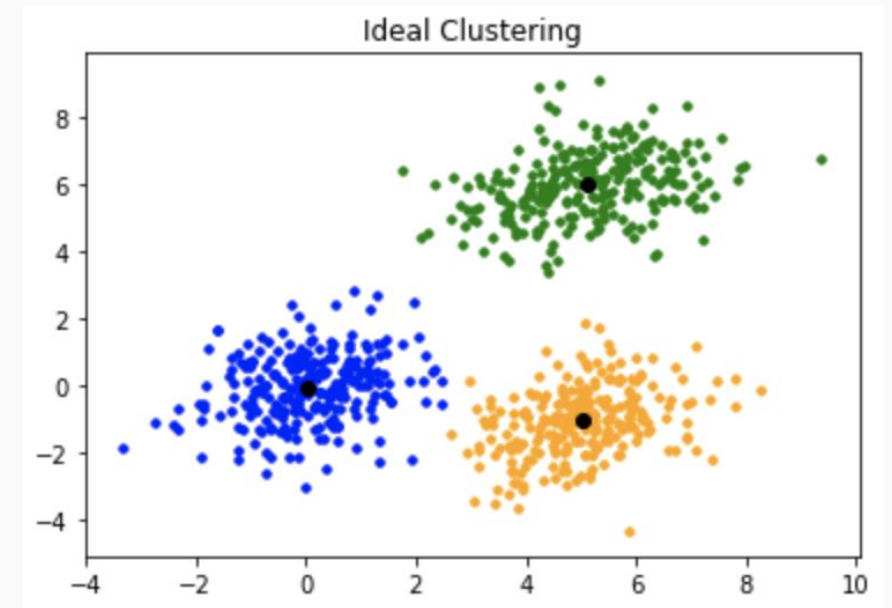
# ¿Qué es K-Means?

Es un algoritmo de clustering particional que divide un conjunto de datos en K grupos (clusters), donde cada grupo tiene un centroide y los datos se asignan al cluster más cercano a ese centroide.



## Objetivo

El objetivo es minimizar la distancia total entre los puntos y sus respectivos centroides.



# ¿Cómo funciona K-Means?

## ✓ Pasos del algoritmo:

1. Elegir K (número de clústeres)
2. Inicializar K centroides aleatorios
3. Asignar cada punto al centroide más cercano
  - ✓ Se usa distancia euclídea u otra métrica
4. Actualizar cada centroide
  - ✓ Se calcula como el promedio (media) de todos los puntos en su clúster
5. Repetir pasos 3 y 4 hasta que:
  - ✓ Los centroides no cambien significativamente, o
  - ✓ Se alcance un número máximo de iteraciones

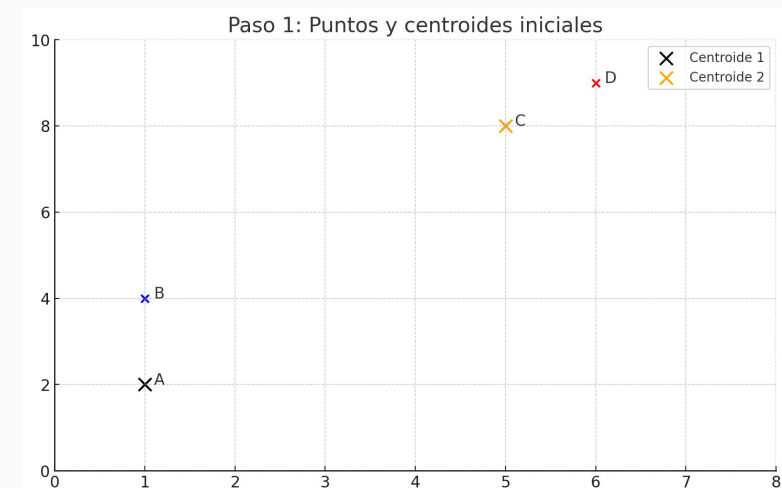
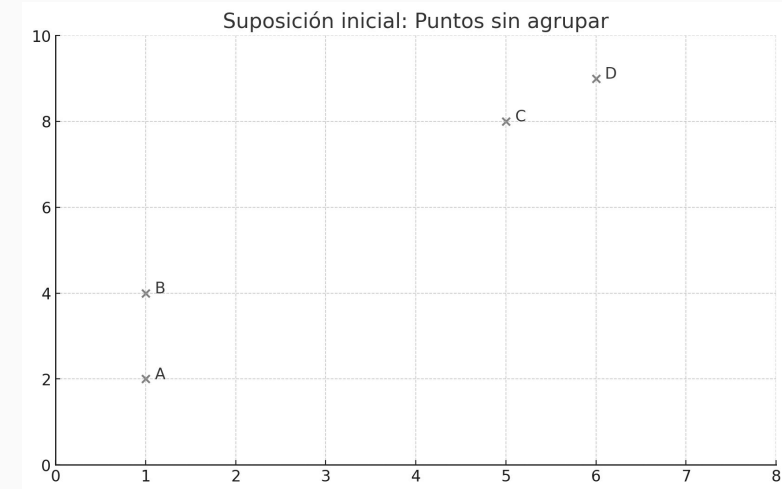
# Por Ejemplo

Supón los siguientes puntos en 2D:

- A(1, 2), B(1, 4), C(5, 8), D(6, 9)
- Queremos hacer K=2 clusters

Paso 1: Inicializamos centroides:

- Cent 1 = A(1, 2)
- Cent 2 = C(5, 8)

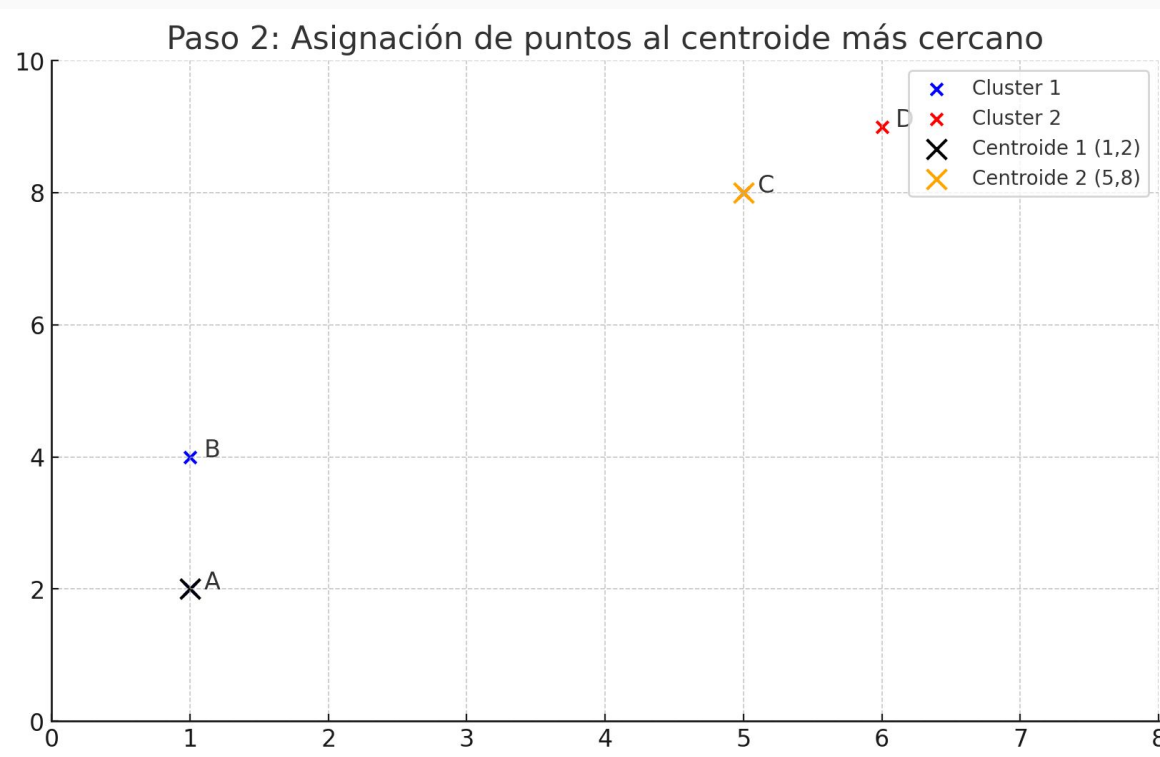


**Paso 2:** Asignamos puntos al centroide más cercano (usando distancia euclídea):

- Distancia A a Cent1 = 0
- Distancia B a Cent1 =  
 $\sqrt{((1 - 1)^2 + (4 - 2)^2)} = \sqrt{4} = 2$
- Distancia C a Cent2 = 0
- Distancia D a Cent2 =  
 $\sqrt{((6 - 5)^2 + (9 - 8)^2)} = \sqrt{2} \approx 1.41$

Resultado:

- Cluster 1: A, B
- Cluster 2: C, D

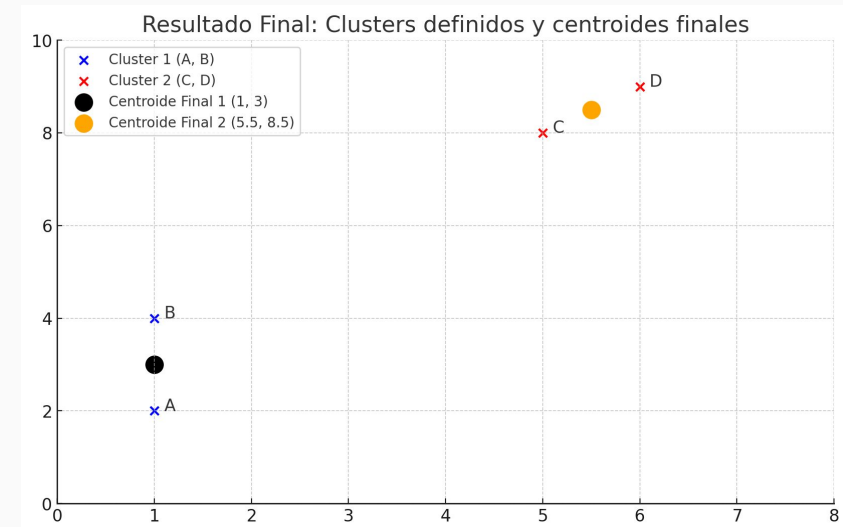
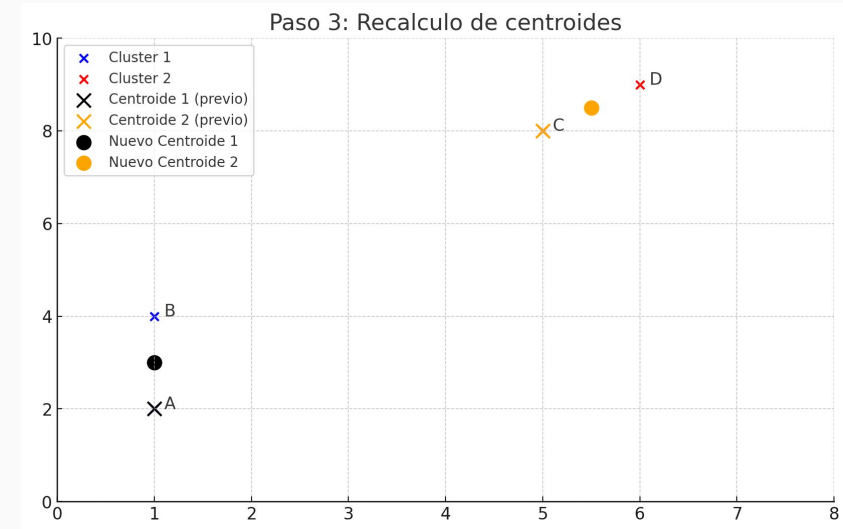


# Por Ejemplo

Paso 3: Calculamos nuevos centroides:

- Cent1 = media(A, B) =  $((1+1)/2, (2+4)/2) = (1, 3)$
- Cent2 = media(C, D) =  $((5+6)/2, (8+9)/2) = (5.5, 8.5)$

Repetimos hasta que los centroides no cambien más.





# K-Means

## ✓ Ventajas

- ✓ • Simplicidad y velocidad
- ✓ • Fácil de implementar e interpretar
- ✓ • Escalable a grandes datasets
- ✓ • Funciona bien con clusters bien separados y esféricos

## ✗ Desventajas

- ❖ Hay que definir K manualmente
- ❖ Sensible a inicialización aleatoria (puede usar k-means++ para mejor inicio)
- ❖ Mal desempeño si los clusters no son esféricos o balanceados
- ❖ Sensible a outliers y escala (es recomendable estandarizar los datos)

# Selección del número óptimo de clusters (K)

Elegir un buen valor para K es crucial:

Un K muy bajo agrupa demasiado los datos (subsegmentación).



Un K muy alto divide demasiado (sobresegmentación).



Y, ¿como sé que número de clúster es el indicado?



Los siguientes métodos ayudan a balancear eso:

- ✓ Método del Codo (Elbow Method)
- ✓ Método de la Silueta

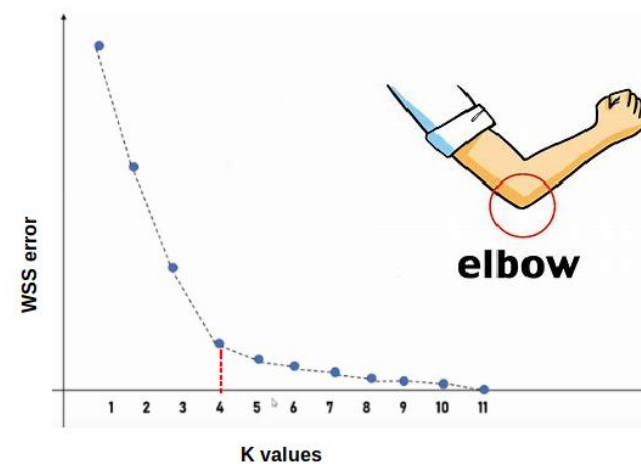


# Método del Codo (Elbow Method)

# ¿Qué es?

Este método busca encontrar el punto donde aumentar el número de clusters ya no mejora mucho la compactación de los mismos.

## Elbow method



Se basa en la curva del Within-Cluster Sum of Squares (WCSS), que mide la suma de las distancias cuadradas entre cada punto y su centroide.

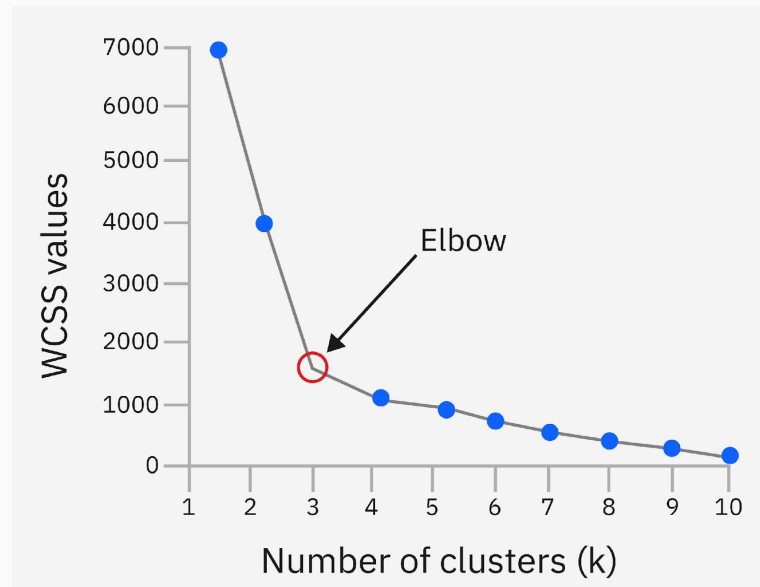
$$WCSS(K) = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2$$

Donde:

- Se calcula WCSS para varios valores de  $K$
- $C_i$  : conjunto de puntos asignados al cluster  $i$
- $\mu_i$  : centroide del cluster  $i$
- $\|x - \mu_i\|^2$  : distancia euclídea al cuadrado entre el punto y el centroide

# Procedimiento paso a paso

1. Ejecuta K-Means para varios valores de K (por ejemplo, de 1 a 10)
2. Calcula el WCSS para cada K
3. Grafica los resultados:
  - ✓ Eje X: número de clusters (K)
  - ✓ Eje Y: WCSS
4. Busca el “codo” de la curva: el punto donde la reducción de WCSS se vuelve lenta





# ¿Cuándo usarlo?

- Cuando se quiere maximizar la compactación del clúster.
- Ideal para datasets donde los clústeres son esféricos y balanceados.



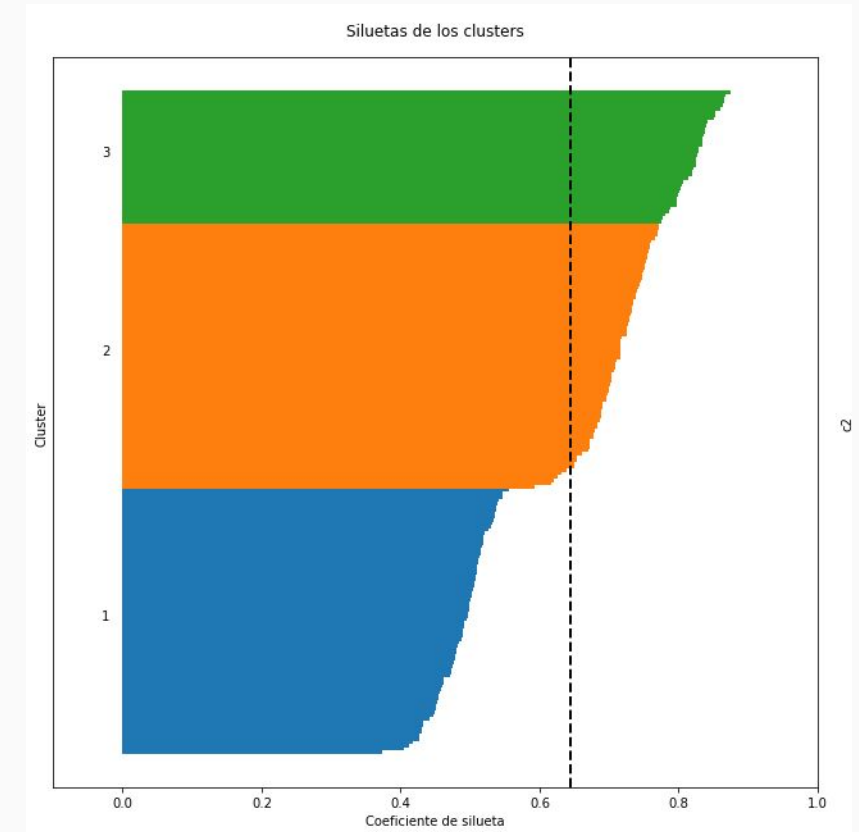
# Método de la Silueta

## ¿Qué es?

El coeficiente de silueta mide qué tan bien separado está un punto de los otros clusters.

Combina:

- La cohesión interna (qué tan cerca están los puntos del mismo grupo)
- Y la separación externa (qué tan lejos están de otros grupos)



Fórmula del coeficiente de silueta para un punto  $i$

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

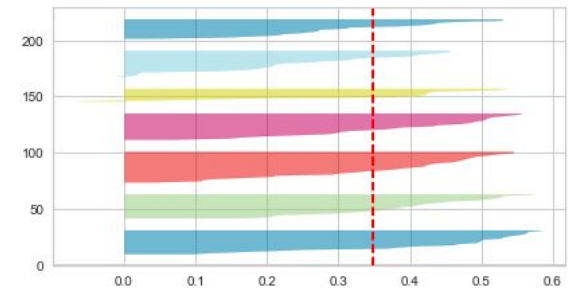
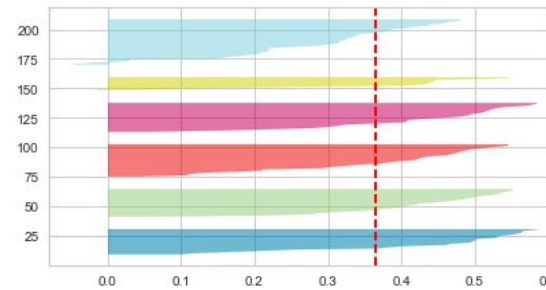
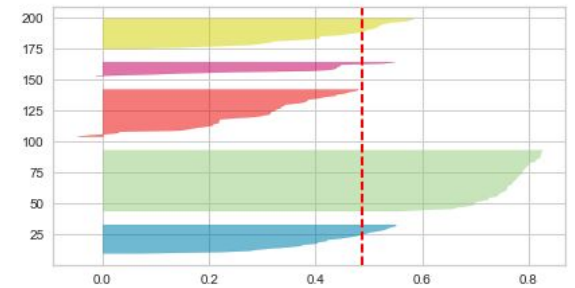
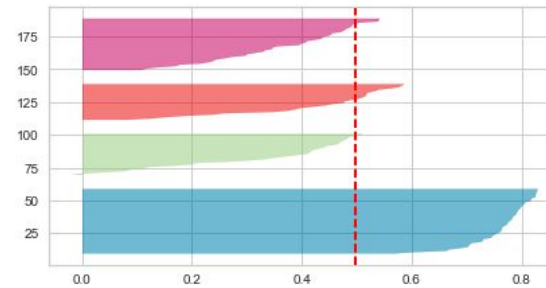
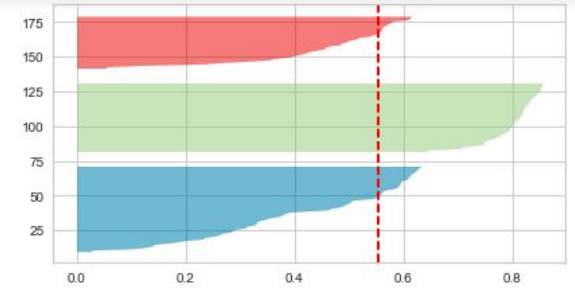
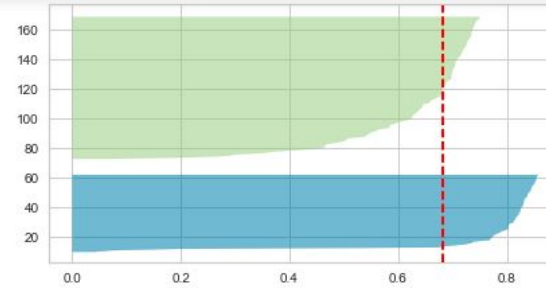
Donde:

- $a(i)$ : distancia media entre  $i$  y los otros puntos de su mismo cluster
- $b(i)$ : distancia media entre  $i$  y los puntos del cluster más cercano (pero distinto)

$$-1 \leq s(i) \leq 1$$

# Interpretación de $s(i)$ :

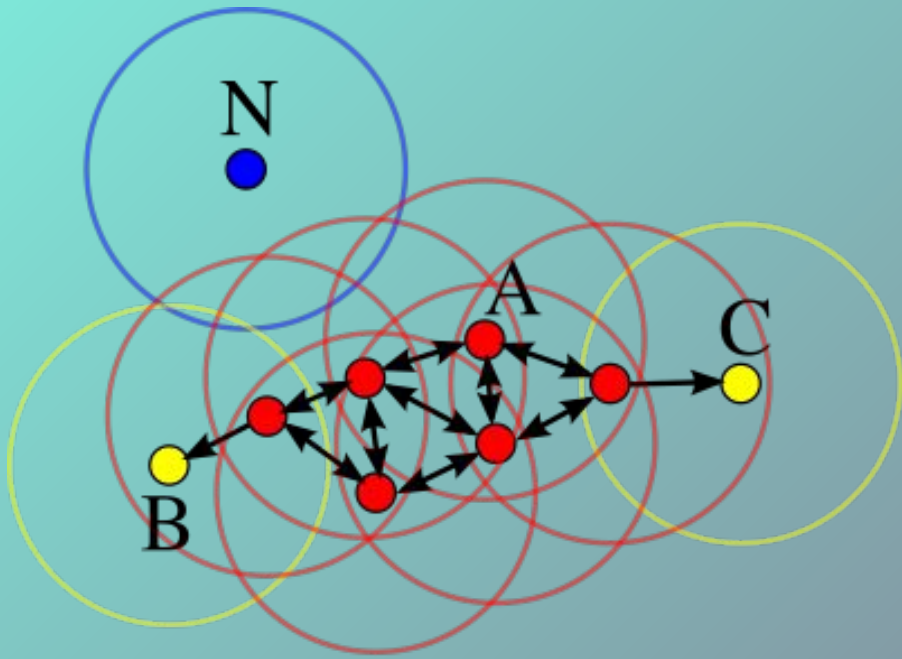
| Valor de $s(i)$    | Significado                                               |
|--------------------|-----------------------------------------------------------|
| Cerca de <b>1</b>  | Buen agrupamiento, bien separado de otros clusters        |
| Cerca de <b>0</b>  | Está en el borde entre dos clusters                       |
| Cerca de <b>-1</b> | Mal agrupado, posiblemente asignado al cluster incorrecto |



# ¿Cuándo usarlo?

- Cuando los clusters pueden solaparse o no ser esféricos
- Complementa muy bien al método del codo
- Es más cuantitativo, ya que tiene un score

| Método  | ¿Qué mide?                  | ¿Cuándo usarlo?                     |
|---------|-----------------------------|-------------------------------------|
| Codo    | Reducción del error interno | Clusters bien definidos y compactos |
| Silueta | Separación y cohesión       | Datasets más complejos o solapados  |



# DBSCAN

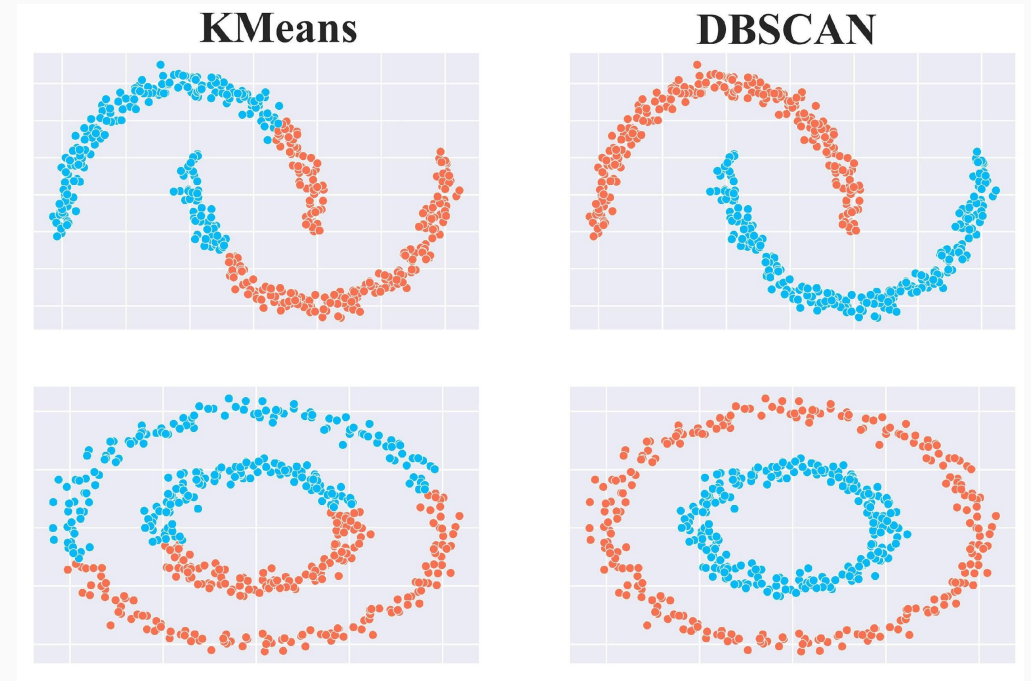
(Density-Based Spatial Clustering of  
Applications with Noise)



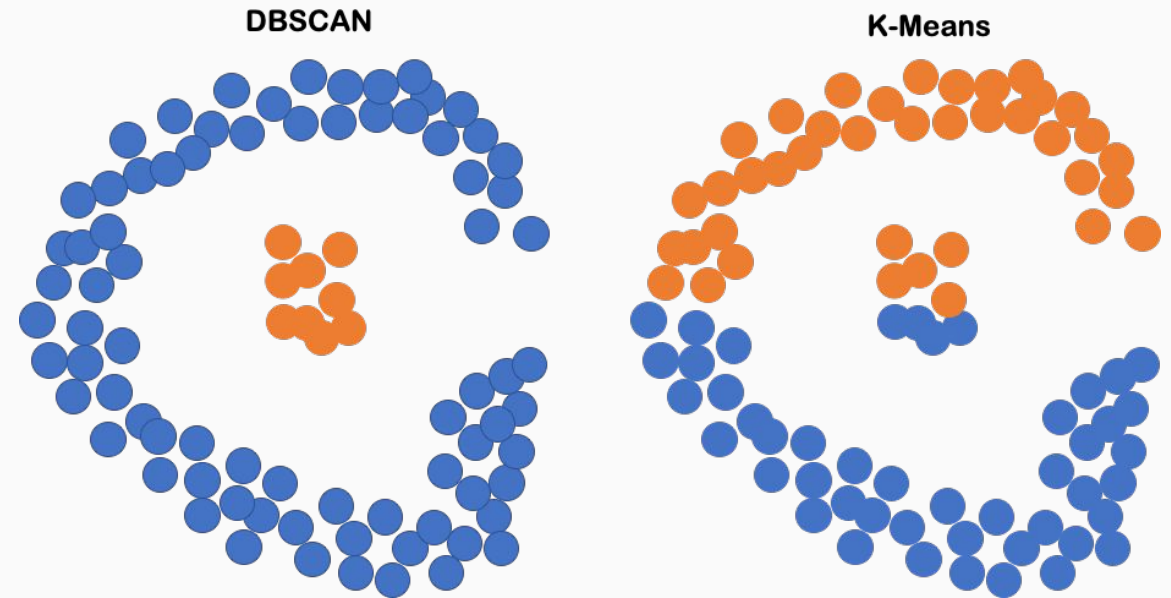
# ¿Qué es DBSCAN?

DBSCAN es un algoritmo de clustering basado en densidad.

En lugar de asumir que los clusters son esféricos o que hay que definir el número de grupos como en K-Means, DBSCAN agrupa puntos que están densamente conectados y detecta automáticamente los outliers como ruido.



No se necesita definir el número de clusters; el algoritmo los encuentra en función de la densidad local de los datos.



# ¿Cuál es la idea principal?

- Puntos en zonas densas → forman clusters
- Puntos en zonas poco densas → se ignoran como ruido
- Dos puntos pertenecen al mismo cluster si están cerca uno del otro y rodeados de muchos otros puntos

# Terminologías clave

| Término              | Significado                                                                                             |
|----------------------|---------------------------------------------------------------------------------------------------------|
| $\epsilon$ (epsilon) | Radio de vecindad: define qué tan cerca deben estar dos puntos para considerarse vecinos                |
| MinPts               | Mínimo de puntos que deben existir dentro del radio $\epsilon$ para que un punto sea considerado núcleo |
| Punto núcleo         | Punto que tiene al menos MinPts vecinos dentro de $\epsilon$                                            |
| Punto frontera       | Tiene menos de MinPts, pero está dentro de $\epsilon$ de un núcleo                                      |
| Punto ruido          | No es núcleo ni frontera; se ignora (outlier)                                                           |

# Parámetros principales

| Parámetro   | Significado                                           |
|-------------|-------------------------------------------------------|
| eps         | Distancia máxima entre dos puntos para ser vecinos    |
| min_samples | Mínimo de puntos en la vecindad para formar un núcleo |

# ¿Cómo funciona DBSCAN?

1. Elige un punto no visitado
2. Si tiene al menos MinPts dentro del radio  $\epsilon$ , es un punto núcleo → se inicia un nuevo cluster
3. Todos los puntos dentro de su vecindad también se agregan al cluster (si también son núcleos, se expanden)
4. Los puntos que no cumplen con los requisitos de densidad, se marcan como ruido
5. Repite hasta recorrer todos los puntos

# No hay fórmula única como en K-Means, pero:

- Se basa en distancias entre puntos:

$$\text{dist}(x_i, x_j) < \varepsilon$$

- La densidad en una zona se mide contando el número de puntos vecinos dentro de ese radio
- No minimiza una función explícita como K-Means

# DBSCAN

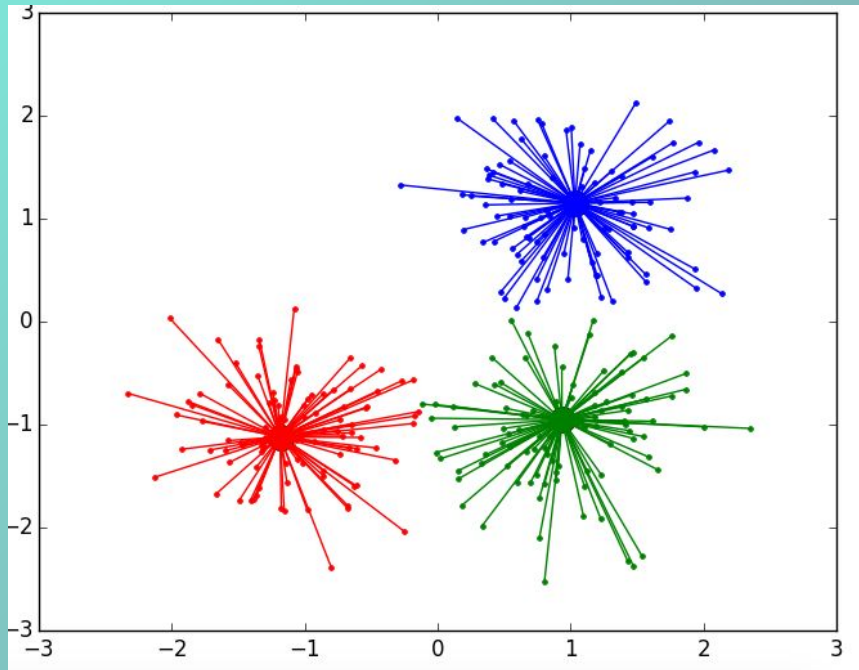
## ✓ Ventajas

- ✓ No necesitas especificar el número de clusters
- ✓ Detecta clusters de forma arbitraria (no esféricos)
- ✓ Maneja ruido y outliers de forma nativa
- ✓ Funciona bien incluso con distribución desigual de datos

## ✗ Desventajas

- ❖ Sensibilidad a los parámetros eps y min\_samples
- ❖ No escala bien con grandes volúmenes de datos (problemas de rendimiento con muchos puntos)
- ❖ No funciona bien si los clusters tienen densidades muy distintas
- ❖ Difícil de ajustar para datos de alta dimensión (curse of dimensionality)



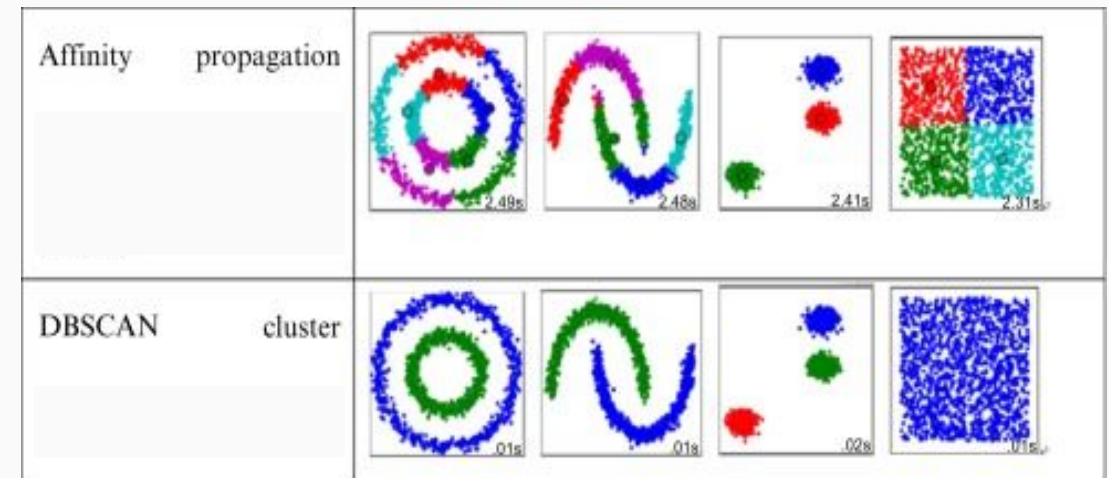


# Affinity Propagation

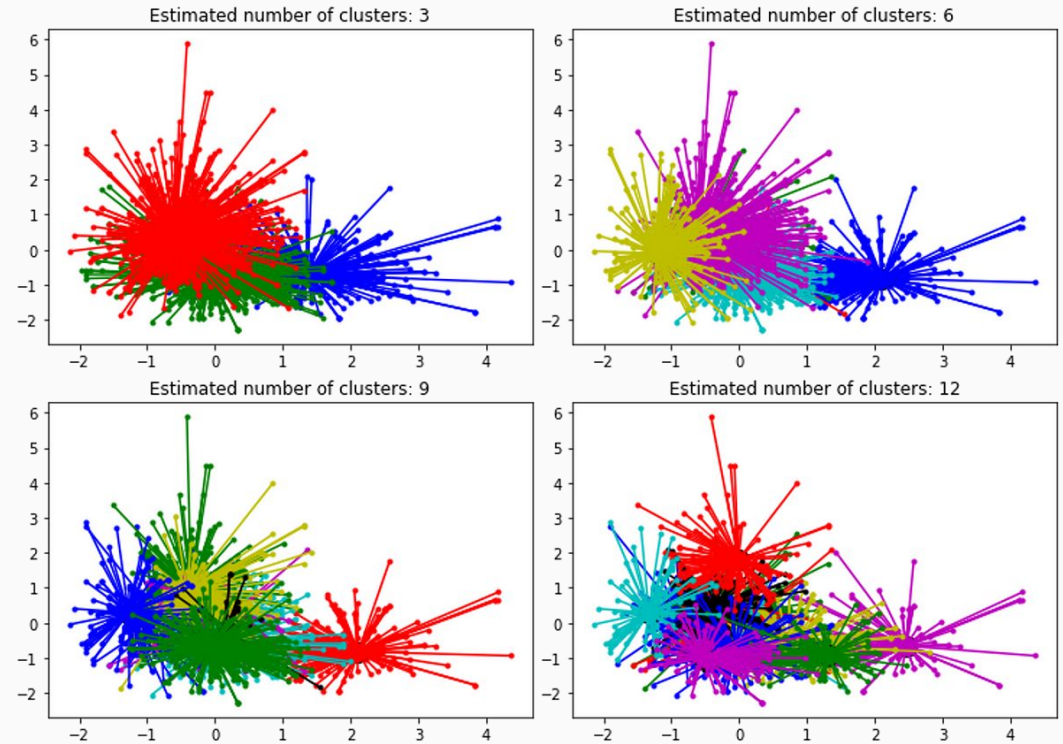
# ¿Qué es Affinity Propagation?

Affinity Propagation (AP) es un algoritmo de clustering basado en similitud que, a diferencia de K-Means o DBSCAN, no necesita definir el número de clusters.

En lugar de elegir aleatoriamente centros de clusters, identifica automáticamente los "ejemplares" (exemplars): puntos representativos que actuarán como centro del cluster.



El algoritmo funciona intercambiando mensajes entre pares de puntos para decidir cuáles serán los ejemplares y cómo se organizarán los clusters.



# ¿Cómo se diferencia de otros algoritmos?

| Comparación          | Diferencia clave                                                          |
|----------------------|---------------------------------------------------------------------------|
| K-Means              | Necesita número de clusters; usa promedios                                |
| DBSCAN               | Basado en densidad; necesita eps y min_samples                            |
| Affinity Propagation | <b>No requiere K</b> ni densidad; los clusters emergen de las similitudes |

# Conceptos clave del algoritmo

| Concepto               | Explicación                                                                                                            |
|------------------------|------------------------------------------------------------------------------------------------------------------------|
| <b>Ejemplar</b>        | Punto representativo del cluster (no es un centroide promedio, sino un punto real)                                     |
| <b>Similitud</b>       | Qué tan adecuados son dos puntos para pertenecer al mismo cluster (generalmente: $-$ distancia euclídea <sup>2</sup> ) |
| <b>Preferencia</b>     | Qué tan probable es que un punto se convierta en ejemplar (por defecto: mediana de todas las similitudes)              |
| <b>Responsabilidad</b> | Mensaje que un punto envía a un posible ejemplar sobre su adecuación                                                   |
| <b>Disponibilidad</b>  | Mensaje que un ejemplar envía sobre su "disponibilidad" para representar otros puntos                                  |

# ¿Cómo funciona?

1. Se define una matriz de similitud entre todos los puntos (puede ser  $-distancia^2$ ).
2. Se inicializan las preferencias de cada punto (autosemejanza).
3. Se realiza un proceso iterativo donde los puntos se comunican:
  - Cada punto sugiere a quién elegir como ejemplar (responsabilidad).
  - Cada candidato decide si está disponible para representar a otros (disponibilidad).
4. Después de varias iteraciones, los puntos convergen a un conjunto de ejemplares y clusters asociados.

# Parámetros principales

| Parámetro        | Significado                                                                 |
|------------------|-----------------------------------------------------------------------------|
| preference       | Define cuántos clusters se formarán (más alto = más clusters)               |
| damping          | Controla la velocidad de convergencia (entre 0.5 y 1.0, evita oscilaciones) |
| max_iter         | Número máximo de iteraciones                                                |
| convergence_iter | Número de iteraciones sin cambios para declarar convergencia                |
| affinity         | Tipo de similitud ('euclidean', 'precomputed')                              |

# Por Ejemplo

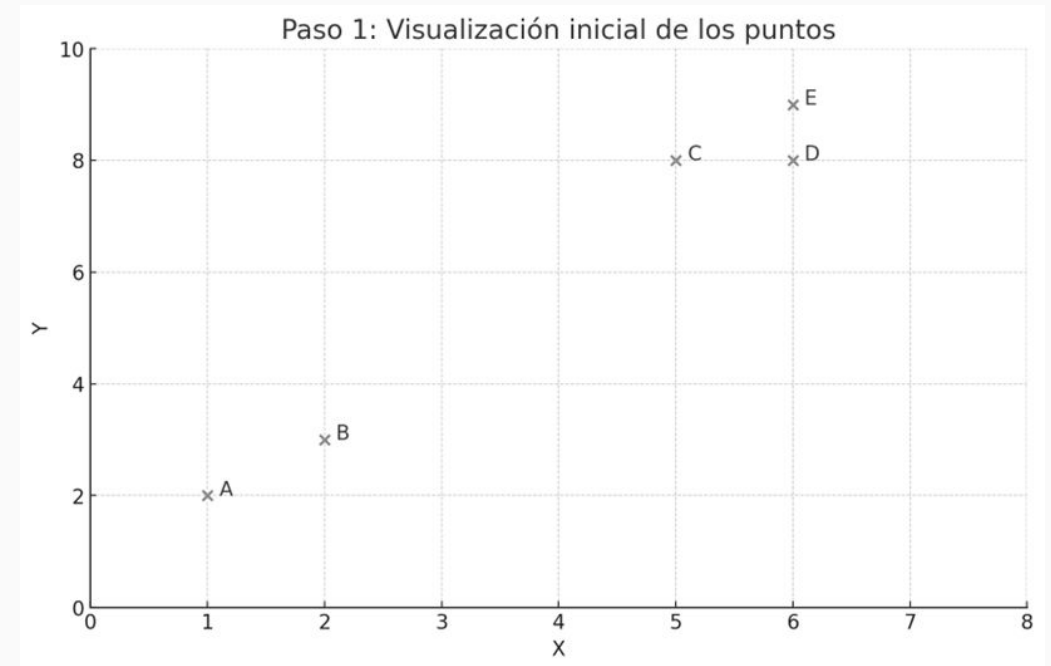
Supón los siguientes puntos en 2D:

| Punto | Coordenadas |
|-------|-------------|
| A     | (1, 2)      |
| B     | (2, 3)      |
| C     | (5, 8)      |
| D     | (6, 8)      |
| E     | (6, 9)      |

Visualmente, parece que hay dos grupos:

Uno con A y B

Otro con C, D y E





# Por Ejemplo

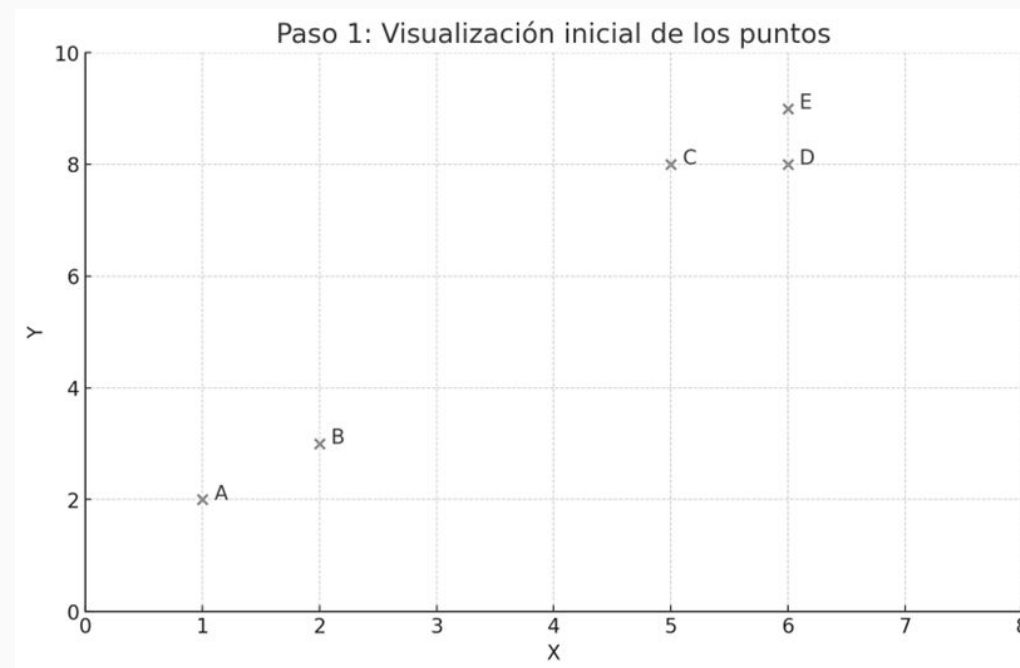
**Paso 1:** Calcular similitud entre puntos

Affinity Propagation no usa distancia directamente, sino similitud.

La similitud estándar es:

$$sim(i, j) = -\|x_i - x_j\|^2$$

(Entre más cercanos estén dos puntos, mayor será su similitud — porque será menos negativa.)



# Por Ejemplo

Creamos una matriz de similitud entre todos los puntos, donde las entradas serán los valores negativos de la distancia euclídea al cuadrado.

Por ejemplo:

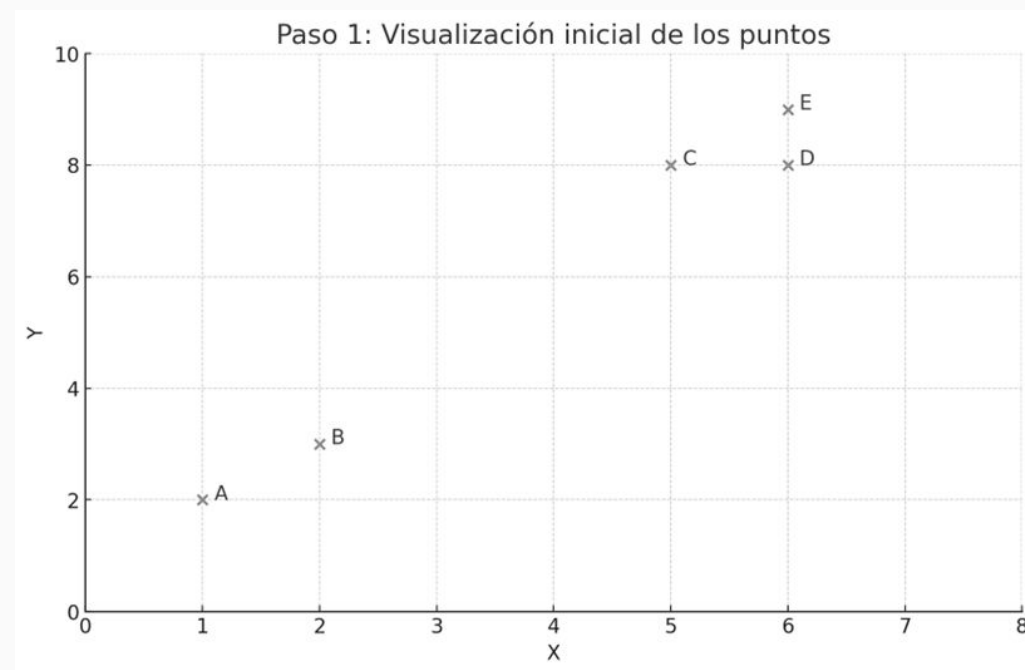
$$\text{sim}(A, B) \approx -((1-2)^2 + (2-3)^2) = -2$$

$$\text{sim}(A, C) \approx -((1-5)^2 + (2-8)^2) = -52$$

$$\text{sim}(C, D) = -((5-6)^2 + (8-8)^2) = -1$$

$$\text{sim}(C, E) = -((5-6)^2 + (8-9)^2) = -2$$

Y así con todos los pares.



## Paso 2: Establecer preferencias

- Cada punto tiene una preferencia de ser elegido como ejemplar (centro).
- Por defecto, se puede usar la mediana de todas las similitudes.
- Esto controla cuántos clusters se formarán:
  - Preferencias altas → más ejemplares → más clusters.
  - Preferencias bajas → menos clusters.

|   | A     | B     | C     | D     | E     |
|---|-------|-------|-------|-------|-------|
| A | -0.0  | -2.0  | -52.0 | -61.0 | -74.0 |
| B | -2.0  | -0.0  | -34.0 | -41.0 | -52.0 |
| C | -52.0 | -34.0 | -0.0  | -1.0  | -2.0  |
| D | -61.0 | -41.0 | -1.0  | -0.0  | -1.0  |
| E | -74.0 | -52.0 | -2.0  | -1.0  | -0.0  |

Supón que usamos la mediana y resulta en que 2 puntos se destacan como buenos candidatos a ejemplar.

## Paso 3: Intercambio de mensajes

El algoritmo ahora comienza un proceso de mensajería entre puntos, que se repite durante varias iteraciones:

### ● Responsabilidad:

- Cada punto sugiere a qué ejemplar le gustaría pertenecer
- Se basa en cuán similar es ese ejemplar a él, en comparación con otros

### ● Disponibilidad:

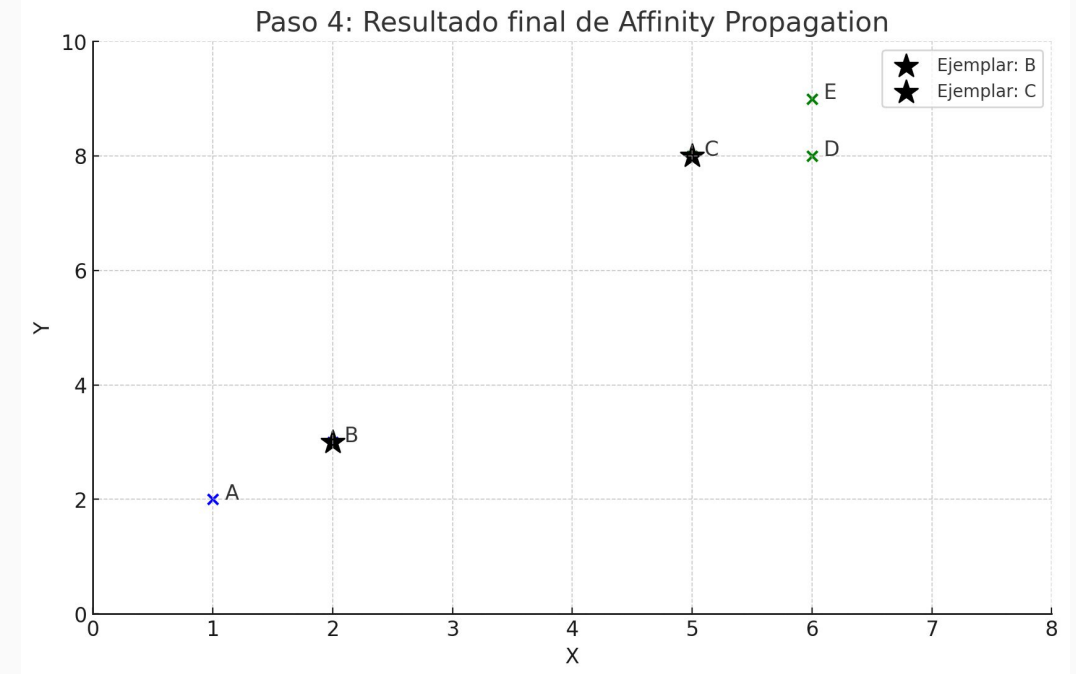
- Cada punto candidato a ejemplar decide si está disponible para representar a otro
- Se basa en cuántos otros puntos lo están "pidiendo" como ejemplar

Este proceso permite que poco a poco los exemplars reales se destaquen.

## Paso 4: Formación de clusters

Después de unas 100 iteraciones:

- ✓ Punto B es elegido como ejemplar para A y B → Cluster 1
- ✓ Punto C es elegido como ejemplar para C, D y E → Cluster 2



# ¿Cuándo usar Affinity Propagation?



Cuando:

- ☐ No sabes cuántos clusters hay
- ☐ Tienes una matriz de similitud propia
- ☐ Quieres un clustering basado en ejemplares reales



Evítalo si:

- ☐ Tienes muchos puntos ( $> 10,000$ )
- ☐ Necesitas mucha interpretabilidad o velocidad

# Affinity Propagation

## ✓ Ventajas

- ✓ No necesitas especificar K
- ✓ Detecta clusters de tamaños y formas arbitrarias
- ✓ Produce clusters centrados en puntos reales, no promedios
- ✓ Ideal cuando tienes una matriz de similitud personalizada

## ✗ Desventajas

- ❖ Computacionalmente costoso: trabaja con matrices  $N \times N$ , lo que lo hace inviable con miles de puntos
- ❖ Sensible a la elección de la preferencia
- ❖ Difícil de ajustar intuitivamente (no tan interpretables como K o eps)

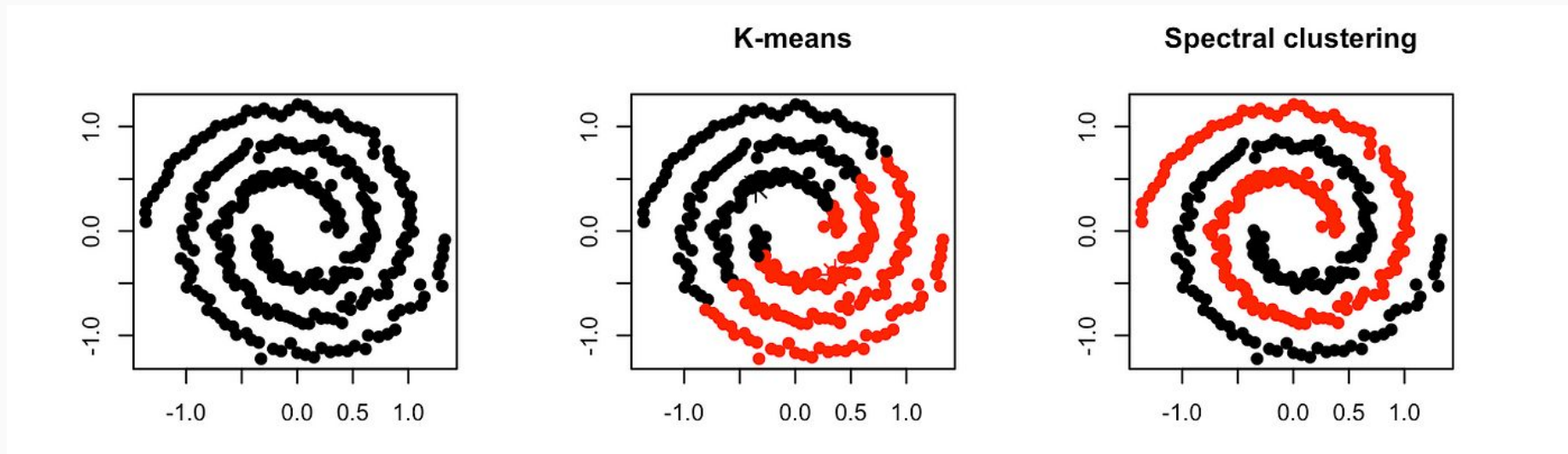


# Spectral Clustering

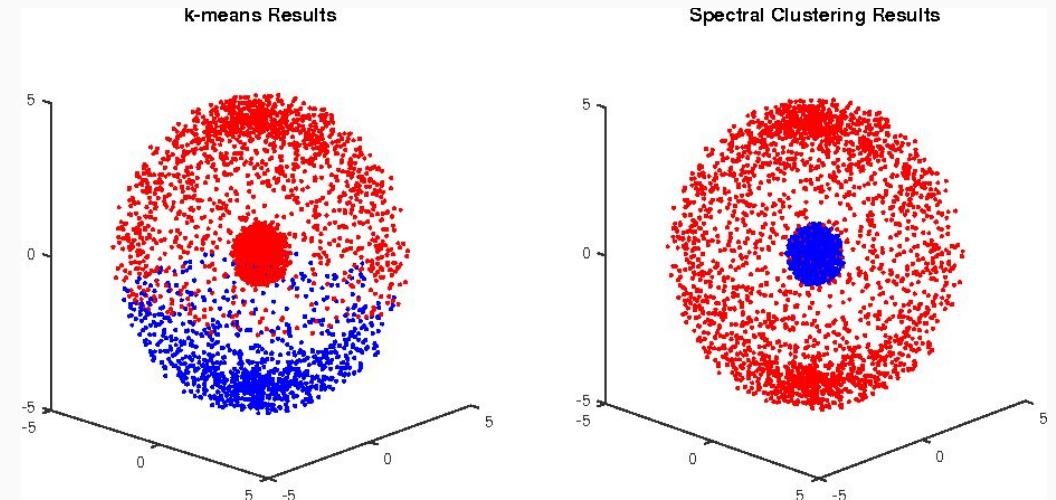


# ¿Qué es Spectral Clustering?

Spectral Clustering es un algoritmo de clustering basado en grafos que utiliza los autovalores y autovectores (espectro) de una matriz de similitud para reducir la dimensionalidad antes de aplicar un algoritmo clásico como K-Means.



En lugar de agrupar los datos directamente, transforma el espacio con ayuda del álgebra lineal (específicamente la descomposición espectral del Laplaciano del grafo) para revelar la estructura de los datos.



# ¿Cuándo se utiliza?

Cuando los datos:

- ✓ Tienen formas complejas que no se pueden separar linealmente
- ✓ No forman clusters esféricos
- ✓ Tienen estructuras de conexión (como datos en forma de luna, espiral, anillos, etc.)

# ¿Cómo funciona?

Paso a paso:

1. Construye una matriz de similitud (por ejemplo, con distancias gaussianas o vecindad k-NN):

$$W_{ij} = \exp\left(\frac{\|x_i - x_j\|^2}{2\sigma^2}\right) \text{ o bien } W_{ij} = 1 \text{ si } x_j \in k\text{-vecinos de } x_i$$

2. Construye el grafo de similitud con esa matriz  $W$
3. Calcula el Laplaciano del grafo:

$$L = D - W \text{ o bien } L_{norm} = I - D^{-1/2} W D^{-1/2}$$

Donde  $D$  es la matriz diagonal de grados del grafo:

$$D_{ii} = \sum_j W_{ij}$$

# ¿Cómo funciona?

Paso a paso:

4. Descomposición espectral:

- Obtén los  $k$  autovectores asociados a los  $k$  menores autovalores del Laplaciano

5. Aplica K-Means sobre estos autovectores para formar los clusters

# ¿Cuándo usar Spectral Clustering?

## ✓ Ideal cuando:

- ✓ Tienes formas no lineales en los datos
- ✓ Trabajas con grafos o relaciones entre elementos
- ✓ K-Means y DBSCAN no logran separar los datos correctamente

## ⊘ Evítalo cuando:

- Tienes más de 10,000 datos (a menos que uses versiones aproximadas)
- Necesitas clustering en tiempo real o muy rápido

# Spectral Clustering

## ✓ Ventajas

- ✓ Detecta clusters no lineales o de forma compleja
- ✓ No depende de centroides o densidad
- ✓ Más preciso que K-Means en estructuras no esféricas
- ✓ Se puede adaptar a grafos (redes sociales, por ejemplo)

## ✗ Desventajas

- ❖ Costoso computacionalmente (requiere descomposición espectral)
- ❖ No escala bien a datasets grandes (recomendado  $< 10,000$  instancias)
- ❖ Requiere construir y almacenar una matriz  $N \times N$ , lo cual es pesado
- ❖ Debes especificar  $K$  (número de clusters)

¡Muchas Gracias!

